

Labo 01

Général

- Auteur: Fabien Léger
- Cours: PTR, HEIG-VD
- Date: 12.03.2026

Questions

cpu_loop.c

Nombre d'itérations avec simple boucle

```
evl-de1 ~ # ./cpu_loop  
There has been 30398477 iterations in 5 seconds!
```

Nombre d'itérations avec boucle et opération

```
evl-de1 ~ # ./cpu_loop  
There has been 26411502 iterations in 5 seconds!
```

Avoir une opération dans une boucle semble avoir un effet non négligeable, bien que le nombre d'itérations changent beaucoup d'un test à l'autre. Il m'est arrivé de faire plus d'itérations avec l'opération que sans parfois.

Multi-tâche

Signification des symboles de la colonne STAT

```
ps maux
```

En faisant `man ps`, on peut trouver une note sur STAT.

```
D    uninterruptible sleep (usually IO)
I    Idle kernel thread
R    running or runnable (on run queue)
S    interruptible sleep (waiting for an event to complete)
T    stopped by job control signal
t    stopped by debugger during the tracing
W    paging (not valid since the 2.6.xx kernel)
X    dead (should never be seen)
Z    defunct ("zombie") process, terminated but not reaped by its parent
...
<    high-priority (not nice to other users)
N    low-priority (nice to other users)
L    has pages locked into memory (for real-time and custom IO)
s    is a session leader
l    is multi-threaded (using CLONE_THREAD, like NPTL pthreads do)
+    is in the foreground process group
```

Voici les symboles qu'on peut découvrir sur la DE1-SoC.

Symbole	État du processus	Description
-	Aucun état / non défini	Pas de signification utile
S	Interruptible sleep	Processus en attente pouvant être réveillé (ex: IO)
Ss	Interruptible sleep and session leader	Processus en attente et leader de sessions c'est-à-dire un shell par exemple
I	Idle kernel thread	Thread du noyau inactif
I<	Idle kernel thread with high-priority	Thread du noyau inactif avec priorité élevée (critique)
R+	Running in foreground	Processus en cours d'exécution et dans le groupe premier plan (terminal actif)

Ordonnancement en temps partagé

```
evl-de1 ~ # ./cpu_loop & ./cpu_loop
There has been 14968083 iterations in 5 seconds!
There has been 15033775 iterations in 5 seconds!
```

```
evl-de1 ~ # ./cpu_loop & ./cpu_loop & ./cpu_loop
There has been 10231902 iterations in 5 seconds!
There has been 10234003 iterations in 5 seconds!
There has been 10234724 iterations in 5 seconds!
```

```
evl-de1 ~ # ./cpu_loop & ./cpu_loop & ./cpu_loop & ./cpu_loop & ./cpu_loop
There has been 6400373 iterations in 5 seconds!
There has been 6419859 iterations in 5 seconds!
There has been 6421242 iterations in 5 seconds!
There has been 6420270 iterations in 5 seconds!
There has been 6421610 iterations in 5 seconds!
```

Il semble que les cinq secondes sont réparties de manière équitable entre les processus. En effet, tous les processus finissent en même temps au bout de cinq secondes et ont un nombre d'itérations similaire. Ce nombre d'itérations est semblable à celui qu'on a eu avec un seul processus, divisé par le nombre de processus lancé. Cela est dû à un ordonnanceur qui donne un temps équitable à nos tâches.

Double terminal

```
evl-de1 ~ # sleep 1; ./cpu_loop & ./cpu_loop
PID149 There has been 8845825 iterations in 5 seconds!
PID150 There has been 8864887 iterations in 5 seconds!
```

```
evl-de1 ~ # ./cpu_loop & ./cpu_loop
PID148 There has been 7874261 iterations in 5 seconds!
PID147 There has been 7932190 iterations in 5 seconds!
```

On voit ainsi que les commandes sans le sleep semblent avoir moins d'itérations. Cela est plutôt étrange, car on s'attendrait à avoir le même nombre d'itérations, dès lors que le temps partagé par les deux processus est de 4s. Peut-être que malgré le sleep, du temps CPU est quand même alloué aux processus alors qu'une fois que les processus sans le sleep sont finis, ceux qui ont attendus 1s ont ainsi du temps avec un processeur plus libre qu'avant.

Migration de tâches

```
evl-de1 ~ # ./cpu_loop
PID186 There has been 14557811 iterations in 5 seconds!
```

```
evl-de1 ~ # ./get_cpu_number
0:38:45 : cpu started on 0
```

Notre logiciel `get_cpu_number` prend du temps CPU, mais ne change pas de processeur. Cela est plutôt logique, car on l'a forcé à agir sur le CPU 0.

En modifiant les variables d'environnement pour que le cpu1 soit utilisable. (fausse sur la donnée)

```
setenv bootargs "console=ttyS0,115200 root=/dev/mmcblk0p3 rw rootwait
earlyprintk"
```

On peut voir le changement de CPU en lançant des programmes sur le CPU correspondant.

```
evl-de1 ~ # ./get_cpu_number
0:18:21 : cpu started on 1
0:18:28 : cpu changed to 0
0:18:31 : cpu changed to 1
0:18:34 : cpu changed to 0
0:18:39 : cpu changed to 1
```

Priorités et niceness

```
evl-de1 ~ # nice -n 5 ./cpu_loop & ./cpu_loop
PID183 There has been 24764257 iterations in 5 seconds!
PID182 There has been 7091818 iterations in 5 seconds!
```

`nice` - run a program with modified scheduling priority

On a que la priorité la plus basse (jusqu'à -20) est priorisée par rapport à la plus haute (jusqu'à +19). Ici, on modifie la priorité du premier processus en l'augmentant de 5. Cela veut dire que le deuxième processus aura la priorité et donc s'exécutera plus de temps processeur que le premier processus.

```
evl-de1 ~ # nice -n 19 ./cpu_loop & nice -n -20 ./cpu_loop
PID193 There has been 28356874 iterations in 5 seconds!
PID192 There has been 375945 iterations in 5 seconds!
```

Codage

```
evl-de1 ~ # ./priority 10
Min prio : 1
Max prio : 98

[01] 0 ( 0%)
[11] 0 ( 0%)
[22] 0 ( 0%)
[33] 0 ( 0%)
[44] 0 ( 0%)
[54] 0 ( 0%)
[65] 0 ( 0%)
[76] 0 ( 0%)
[87] 55800518 (50%)
[98] 55810424 (50%)
```

On voit bien que la priorité la plus haute signifie que les programmes s'exécutent. Le problème est que nous n'avons que 2 processeurs qui exécutent donc les priorités les plus hautes. Ainsi, il n'y a au final que 2 processus qui auront eu des itérations alors que les autres se trouvent en famine.

On peut donc noter la différence entre niceness et priorité. Niceness donne une préférence, mais laisse toujours du temps cpu à tous les processus. La priorité, elle, ne donne qu'au processus avec la plus haute priorité.